

Chapter 7 Memory Systems

7.1 Memory and Computer Operation Principles

7.2 Hierarchical memory system

7.3 Types and Structures of Physical Memory

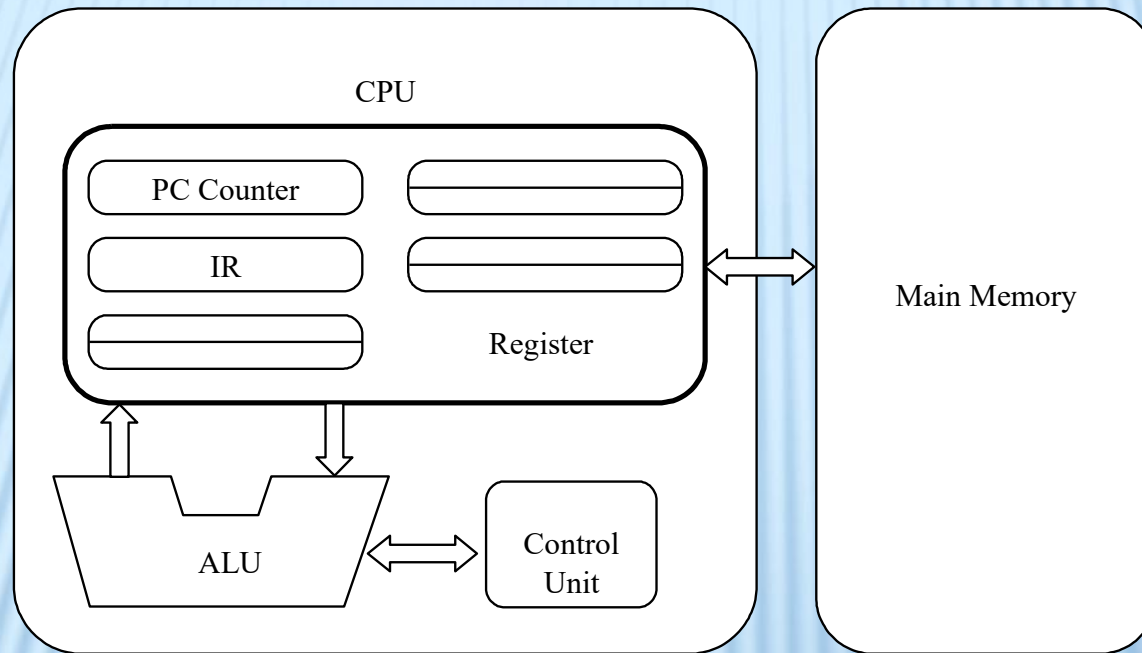
7.4 Cache Memory

7.5 Virtual Memory

7.1 How memory and computers work

- ✖ Since Von Neumann proposed the concept of stored programs, most computers have been built using the memory-centric Von Neumann architecture.
 - + All programs and data that need to be executed must be stored in memory before being processed by the CPU to retrieve instructions and data from the main memory.
 - + Every instruction executed by the CPU and every piece of data processed also comes from memory.

7.1 How memory and computers work



Memory-centric Von Neumann architecture

7.2 Hierarchical memory system

✖ Hierarchical memory system :

- + Computer systems use the combination of different types of memory to solve the problems of "CPU and memory running speed is not equal" and "memory is never large enough".
- + Provide users with high-performance and low-cost enjoyment.

✖ The idea of hierarchical memory :

- + Different memory tiers are divided, each using memory with different access speeds, storage capabilities, and price points, and then arranging the memory tiers where the data is located according to the frequency of access.

7.2 Hierarchical memory system

✖ Components of a hierarchical memory system :

- + Register
- + Two-tier cache
- + Main memory
- + Hard drives, CDs, tapes, secondary memory , (auxiliary memory)

7.2 Hierarchical memory system

Memory hierarchy	Access time	Capacity units	Task level	
register	1ns~2ns	Word	main system	Expensive
The first layer of cache	3ns~10ns	K Byte		
Layer 2 cache	25ns~50ns	M Byte		
main memory	30ns~90ns	G Byte		
hard disk	5ms~20ms	100 G Byte ~ T Byte	Works online	
CD (burner) machine	100ms~5s	Unlimited expansion	Works semi-offline	
Tape drive	10s~3min	Unlimited expansion	Works offline	cheap

Hierarchical memory system

7.2 Hierarchical memory system

- ✖ A way to implement a hierarchical memory system

7.2 Hierarchical memory system

- ✖ In a hierarchical structure, the "distance" between memory and processor determines the time it takes to access it.
 - + It is usually measured in clock cycles.
 - + The closer the memory is to the processor, the fewer read cycles it takes.
 - + The farther away you are from the processor, the more cycles the access will take.

7.3 The type and structure of physical memory

- ✖ Memory can be divided into two categories based on its ability to accept writes :
 - + Random Access Memory , (RAM)
 - + Read-Only Memory (ROM)

7.3 The type and structure of physical memory

- × RAM

- × ROM

7.3.1 RAM

- ✖ RAM is the largest use of RAM :
 - + It is used as the "main memory" of the computer, which is used to store the programs and data required for the operation of the computer.
- ✖ Once the computer is powered off, the data stored in RAM disappears, so it is also called volatile memory (volatile memory).

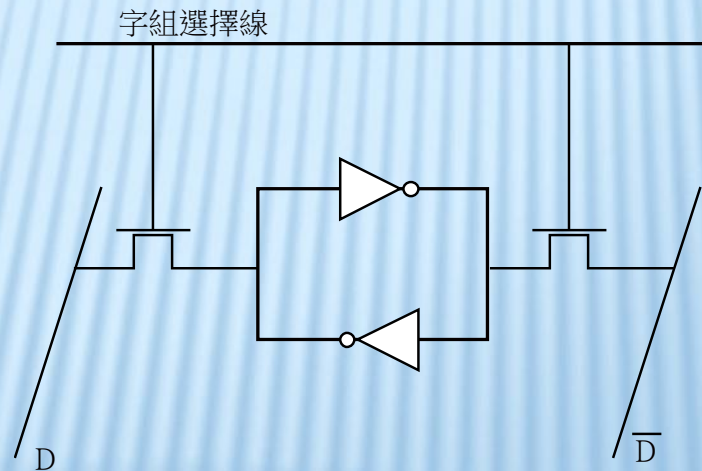
7.3.1 RAM

- ✖ There are two types of RAM in today's computer systems :
 - + Static RAM(SRAM)
 - ✖ The access speed is faster than DRAM
 - ✖ Expensive
 - ✖ as cache memory
 - + Dynamic RAM(DRAM)
 - ✖ The higher density allows for the storage of more bits in the chip
 - ✖ It consumes less power than SRAM
 - ✖ as the main memory

7.3.1 RAM

✖ SRAM

- + A storage device made with semiconductor technology. The storage principle is to use a structure similar to a D-flip-flop.



Simple construction diagram of SRAM

7.3.1 RAM

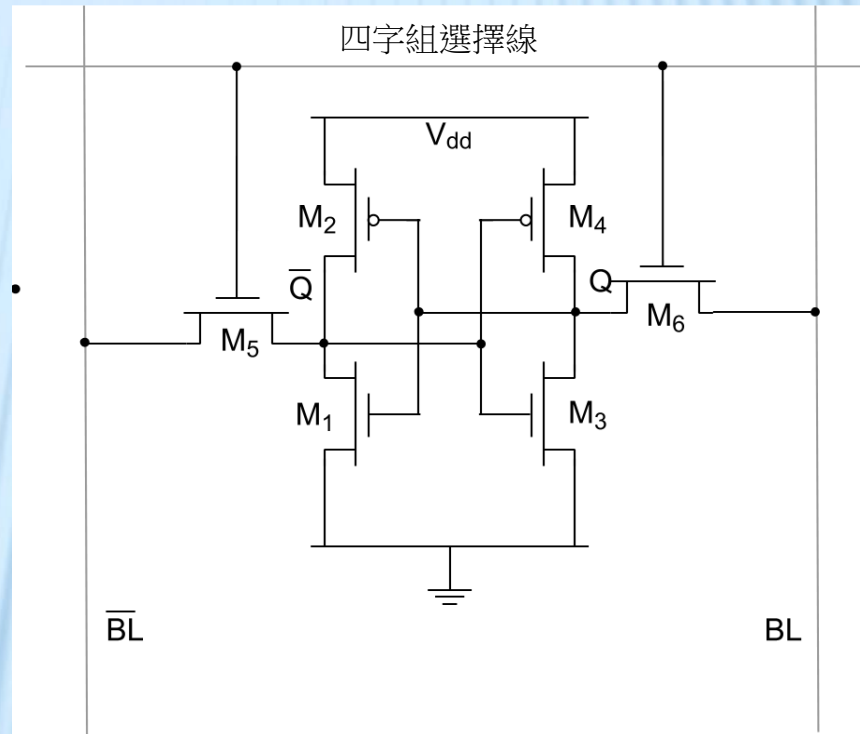
✖ SRAM

- + When data is stored in the component, SRAM can preserve its contents as long as power is continuously supplied.
- + A single bit of SRAM memory requires six transistors, a memory design known as the "6T design".

7.3.1 RAM

✖ SRAM

+ 6T design
architecture



Each SRAM cell requires 6 transistors

7.3.1 RAM

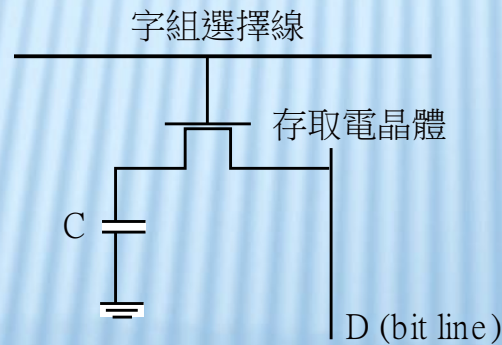
✖ DRAM

- + It is also a type of memory that uses a semiconductor process. The principle of action is mainly to use a tiny capacitor etched on a semiconductor to store the charge, and according to the amount of charge, it is used to represent whether the value of a single bit (bit) is 1 or 0.
- + Since capacitors themselves can leak current, existing memory values must be written back (i.e., recharge or discharge capacitors) continuously and periodically before data loss to ensure data accuracy.

7.3.1 RAM

✖ DRAM

- + After each read, the charge of its capacitor is also lost, so the data stored by it must also be restored through reorganization, which is a destructive operation.



Simple construction diagram of DRAM

7.3.1 RAM

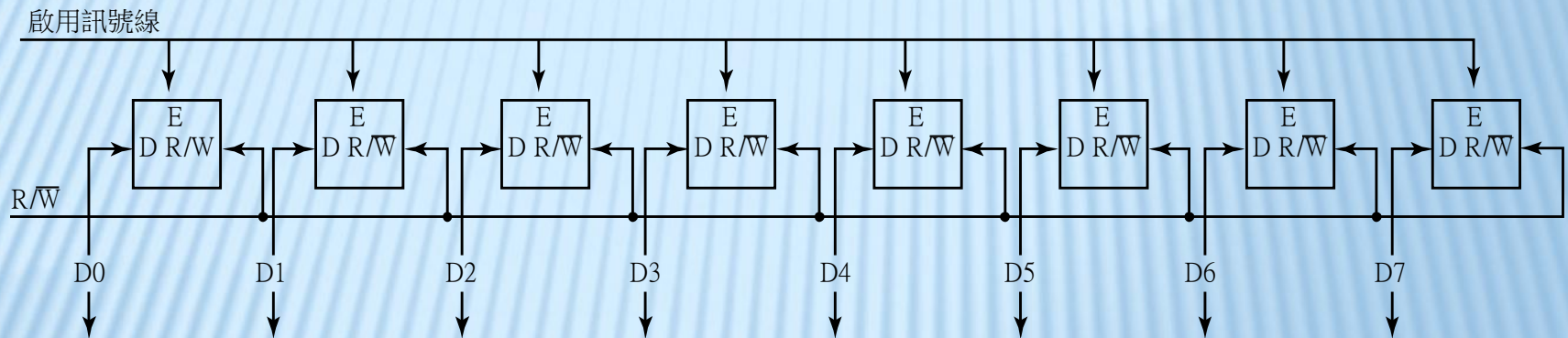
✖ DRAM

- + It has the advantage of simple structure and can have a very high density.
 - ✖ Each bit of data only needs one capacitor and one transistor to process, so it is called "**one T design**".
- + It has the disadvantage of "slow access", and the additional mechanism and runtime required for the retooling operation is considerable.

7.3.1 RAM

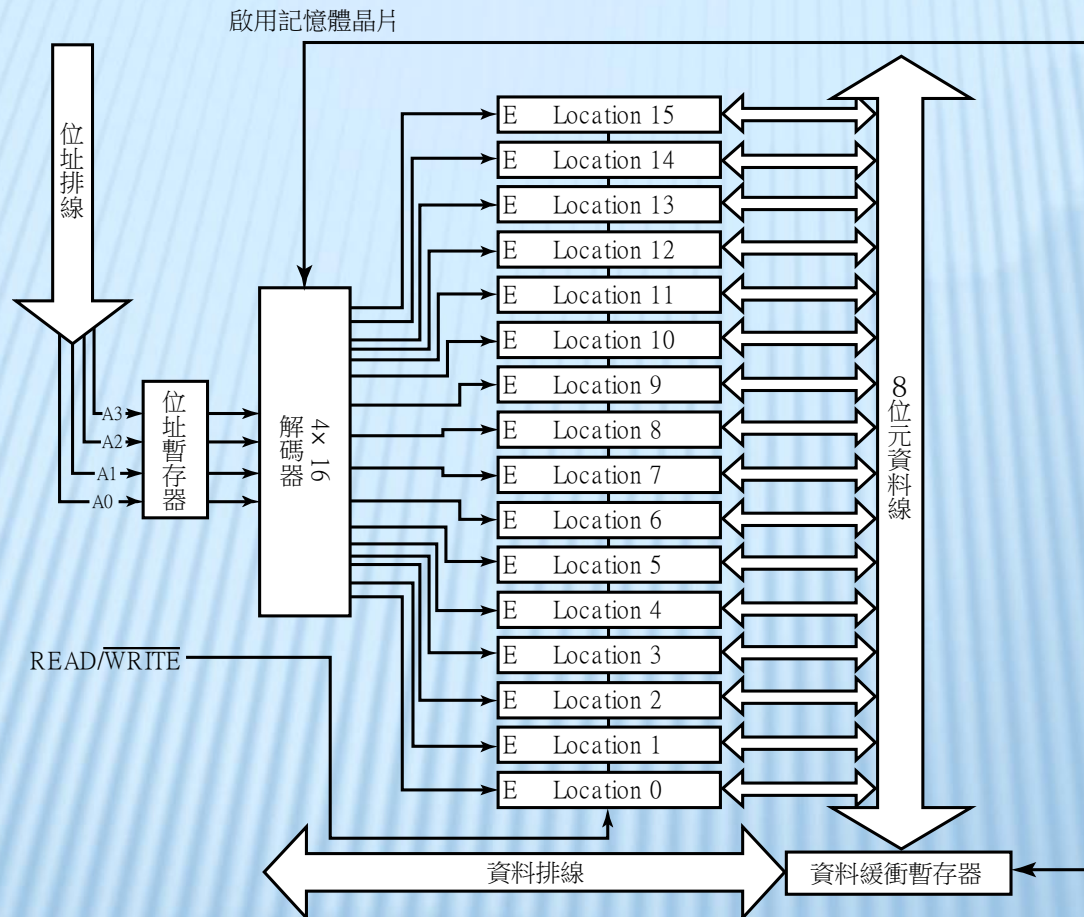
- ✖ The structure of the array of RAM
 - + Eight identical memory bits are arranged in parallel, along with a common control line and separate output/input data lines, to complete a byte memory unit.
 - + Organizing the byte cells, such as arranging them in a linear structure, can form a simple memory chip.

7.3.1 RAM



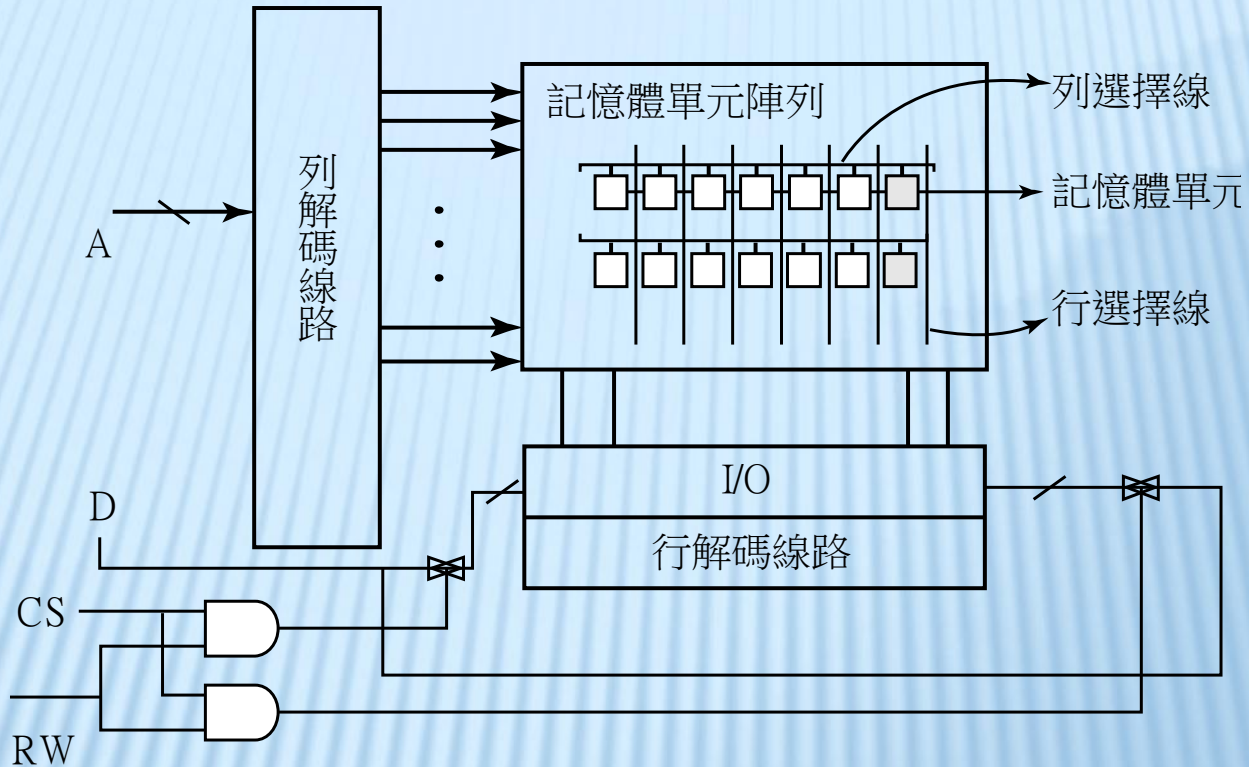
A diagram of a memory unit for one byte

7.3.1 RAM



The array of RAM is actually a matrix of single memory units neatly arranged with control and transmission wiring

7.3.1 RAM



By changing the linear arrangement in a single bit chip to a two-dimensional structure and using the address decoders of rows and rows respectively, the wiring of the entire memory chip can be much simplified.

7.3 The type and structure of physical memory

- × RAM

- × ROM

7.3.2 ROM

- ✖ ROM is another commonly used type of memory that is characterized by:
 - + Data can only be read out and not written directly.
 - + After the computer is turned off, the data will still be saved.
- ✖ The design of ROMs can be divided into five different types :
 - + **ROM 、 PROM 、 EPROM 、 EEPROM 、 Flash Memory**

7.3.2 ROM

memory class	Read speed	Write speed	The number of times it can be written	Volatile memory
ROM	fast	No	0	not
PROM	fast	No	1	not
EPROM	fast	No	many	not
Flash	fast	slow	Approx. 10,000	not
EEPROM	slow	slow	Approx. 1,000,000	not
RAM	Very fast	Very fast	Unlimited times	Yes

Comparison of different types of memory

7.3.2 ROM

✖ ROM

- + In this case, it refers to the most primitive read-only memory, which is hardwired and stores data directly with unchangeable hardware wiring as soon as it leaves the factory.

7.3.2 ROM

✖ PROM

- + Programmable Read-Only Memory, called Programmable Read-Only Memory, is a variant of ROM whose memory unit is composed of some fusible wiring. The user can program the data into the PROM by using a special device, that is, by cutting off part of the fusible wire; However, it only allows writing once, and once it is programmed, the data and instructions in it cannot be changed.

7.3.2 ROM

✖ EPROM

- + The full name is Erasable Read-Only Memory (Erasable PROM), and it is more advanced than PROM in that it already has the ability to write repeatedly. Before writing a new EPROM, it is necessary to use ultraviolet light to irradiate a special window on the chip to clear the existing memory content.

7.3.2 ROM

✖ EEPROM

- + Called Electrically Erasable PROM, it improves many of the shortcomings of EPROM, such as eliminating data without using a special ultraviolet lamp, but instead using an electric field to erase memory content; It can even eliminate a certain part of the chip in bytes.

7.3.2 ROM

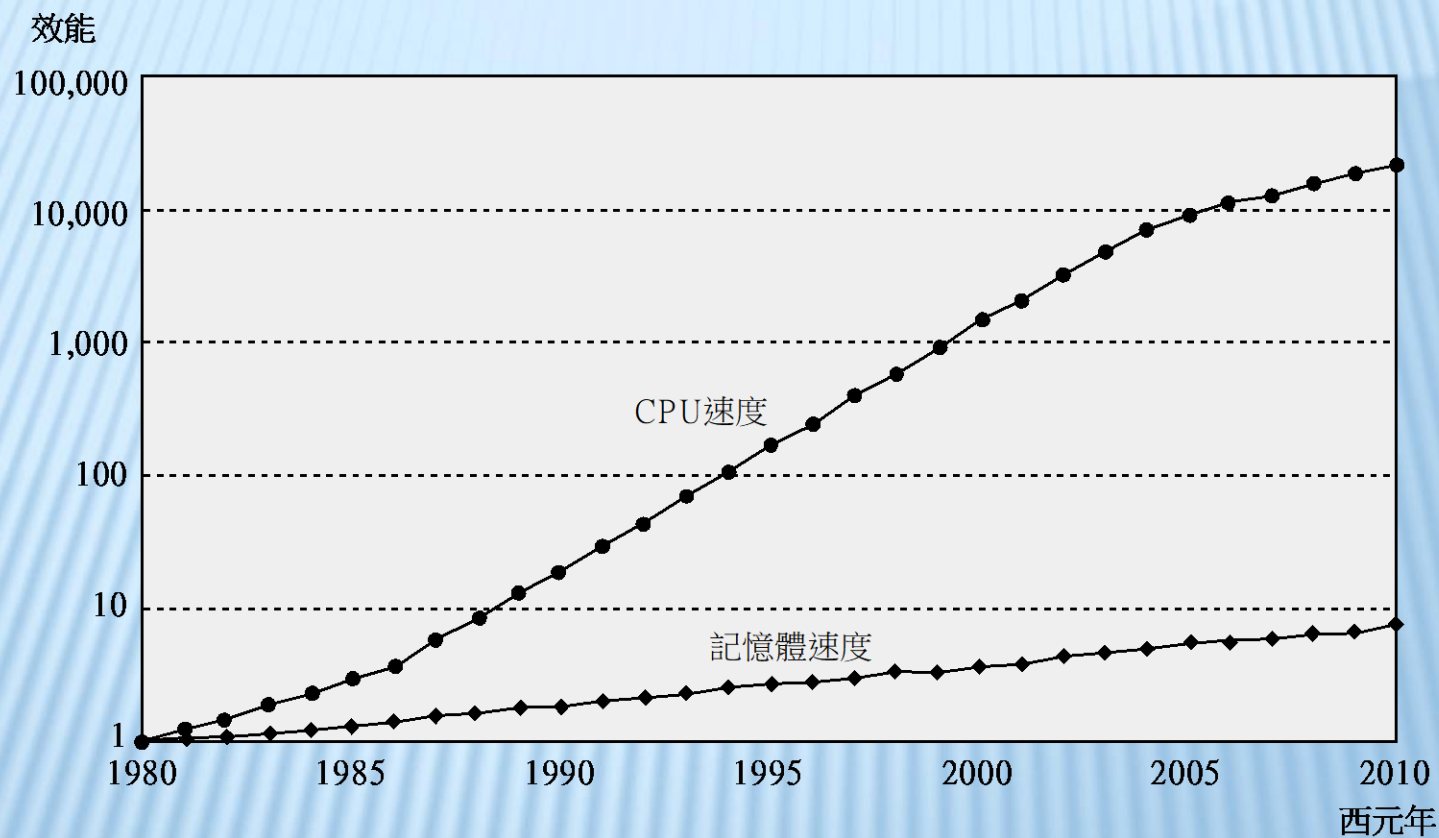
✖ Flash Memory

- + Commonly known as flash memory, sometimes referred to simply as Flash, is essentially an EEPROM that is written or erased in blocks. Because it improves the limit of only one byte at a time, it is much faster than EEPROM in terms of speed. Nowadays, larger flash memory has developed to a considerable extent in terms of both capacity and speed, such as the commonly used thumb disk or the memory card of digital cameras, which uses flash memory, which can reach 32GB in size.

7.4 Cache memory

- ✖ There has long been a significant performance gap between processors and memory, and the speed at which processor computing performance is improving is making it harder for main memory to keep up.
- ✖ Cache architecture :
 - + It is used to close the speed gap between the computer's CPU and memory, so that the processor does not slow down.

7.4 Cache memory



Comparison of the development trend of
computer CPU and memory

7.4 Cache memory

- ✖ The 80/20 rule and the regionality of references
- ✖ First acquaintance with cache memory
- ✖ The basic operating mode of cache memory
- ✖ The mapping mechanism of the cache
- ✖ Replacement strategy
- ✖ Effective access time and hit rate
- ✖ Write strategy for cache memory

7.4.1 The 80/20 rule and the regionality of references

✖ The 80/20 rule :

- + When a program is executed, more than 80% of the time spent on the computer is actually focused on less than 20% of the instructions and data.
- + This phenomenon is called the locality of reference.
- + Because memory has a regional nature of reference, computer systems have the opportunity to effectively accelerate access to programs or data using only a small amount of fast memory.

7.4.1 The 80/20 rule and the regionality of references

- ✖ There are two main basic forms of regionality of reference :
 - + Temporal regionality :
 - ✖ Refers to a memory address that has been accessed recently and is likely to be accessed again soon.
 - + Regionality in space :
 - ✖ Refers to the fact that when a memory address is accessed, its neighboring memory addresses are likely to be accessed. That is, the behavior of accessing has a tendency to "cluster" in memory.

7.4 Cache memory

- ✖ The 80/20 rule and the regionality of references
- ✖ **First acquaintance with cache memory**
- ✖ The basic operating mode of cache memory
- ✖ The mapping mechanism of the cache
- ✖ Replacement strategy
- ✖ Effective access time and hit rate
- ✖ Write strategy for cache memory

7.4.2 First acquaintance with cache memory

- ✖ Cache memory is a small, fast memory that is used as a buffer for more frequently accessed data in the main memory (buffer) ◦
 - + Cache memory consists of SRAM.
 - + The main memory is composed of DRAM.

7.4.2 First acquaintance with cache memory

- ✖ Operating mode of a hierarchical memory system :
 - + The processor first finds the data in the L1 cache.
 - + If the data does not exist in the L1 cache, it is necessary to look for it in the L2 cache at the next layer.
 - + If the data is not yet searched, the data is searched layer by layer to the memory device farther away from the CPU.

7.4.2 First acquaintance with cache memory

✖ Common terms for the operation of hierarchical memory systems :

- + block
- + hit
- + hit rate
- + miss
- + miss rate
- + hit time
- + miss penalty

7.4.2 First acquaintance with cache memory

✖ block

- + The units of data mapping and transmission between the two adjacent layers are mostly words, and the size is the power of 2. Each block has its own number: for example, the number of cache memory is determined by the logical order of the blocks in memory; The number of the main memory block is determined by a specific field in the memory address, that is, a few bits in it.

7.4.2 First acquaintance with cache memory

✖ hit

- + The required word data resides in a given tier of memory (usually only concerned with whether the higher memory level hits or not).

✖ hit rate

- + The probability (percentage) of finding data in a given tier of memory.

[cache_implementation.html](#)

7.4.2 First acquaintance with cache memory

- ✖ miss

- + The required data is not in memory for a given tier.

- ✖ miss rate

- + The percentage of data that cannot be found in a given tier of memory clearly indicates that the error rate = $1 - \text{hit rate}$.

7.4.2 First acquaintance with cache memory

✖ Hit time

- + The processing time it takes to find the required information in a given tier of memory.

✖ Miss penalty

- + The processing time it takes to process a mistake. This time includes the time it takes to transfer blocks from the lower storage to the upper memory, plus the processing time to send the required data to the processor.

Cache Hit/Miss

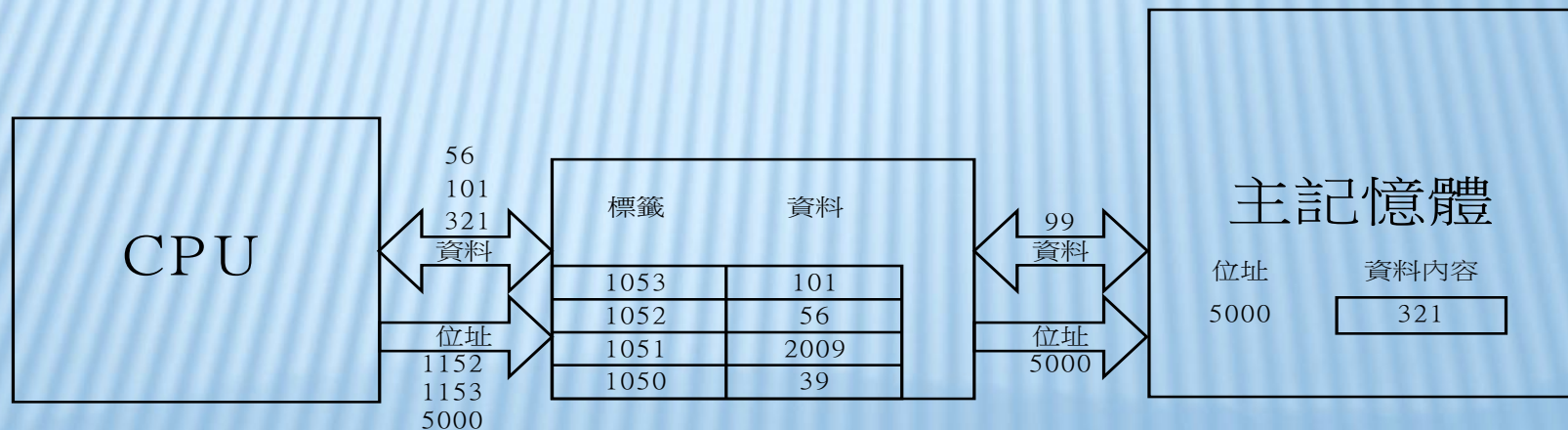
7.4 Cache memory

- ✖ The 80/20 rule and the regionality of references
- ✖ First acquaintance with cache memory
- ✖ **The basic operating mode of cache memory**
- ✖ The mapping mechanism of the cache
- ✖ Replacement strategy
- ✖ Effective access time and hit rate
- ✖ Write strategy for cache memory

7.4.3 The basic operating mode of cache memory

✕ Locate the mapped cache address

- + The mode of operation of cache memory is to store memory data that has just been accessed or may be accessed by the CPU into memory that is fast and "close" to the CPU than the main memory.



Cache memory architecture

7.4.3 The basic operating mode of cache memory

- ✖ Locate the mapped cache address

- + What data is most likely to be accessed in the following time :

- ✖ Using the locality of the reference, each time a memory access operation is performed, it is sent from the main memory to the cache memory in blocks

- + The two factors that determine where the newly sent block should be placed in the cache memory or which cache block the data the CPU wants to access may be mapped to :

- ✖ The amount of cache memory

- ✖ Cache mapping policy

7.4.3 The basic operating mode of cache memory

- ✖ Determine whether the cache content is hit
 - + Determining whether a hit is usually required with some additional information.
 - + The cache must provide a record for each of the blocks within it, with at least two fields :
 - ✖ valid bit
 - ✖ tag field

7.4.3 The basic operating mode of cache memory

Cache block number	valid	tag	data
0000			
0001			
:			
:			
:			
:			
:			

The record of each cache block should have at least these two fields on the left to determine whether the block contains the data we want to access

7.4.3 The basic operating mode of cache memory

✖ Determine whether the cache content is hit

+ Step 1 : Check the significant bits first

- ✖ If the significant bit is 0, it means that the cache block is invalid, and a cache error is generated at this time, and the main memory must be accessed to complete the data access.
- ✖ If the valid bit is 1, it means that the cache block is still valid and may produce cache hits; However, the next step is to check whether the block in the cache is from the block where the target memory is located.

7.4.3 The basic operating mode of cache memory

- ✖ Determine whether the cache content is hit
 - + Step 2 : Match whether the tag field matches the memory address you want to access
 - ✖ A tag field is actually a collection of bits, and the values of these bits come from the main memory address, so they must match to find the required cache block.

7.4 Cache memory

- ✖ The 80/20 rule and the regionality of references
- ✖ First acquaintance with cache memory
- ✖ The basic operating mode of cache memory
- ✖ **The mapping mechanism of the cache**
- ✖ Replacement strategy
- ✖ Effective access time and hit rate
- ✖ Write strategy for cache memory

7.4.4 The mapping mechanism of the cache

- ✖ Direct Mapped Cache
- ✖ Fully-Associative Cache
- ✖ Set Associative Cache

explain the cache memory mapping techniques

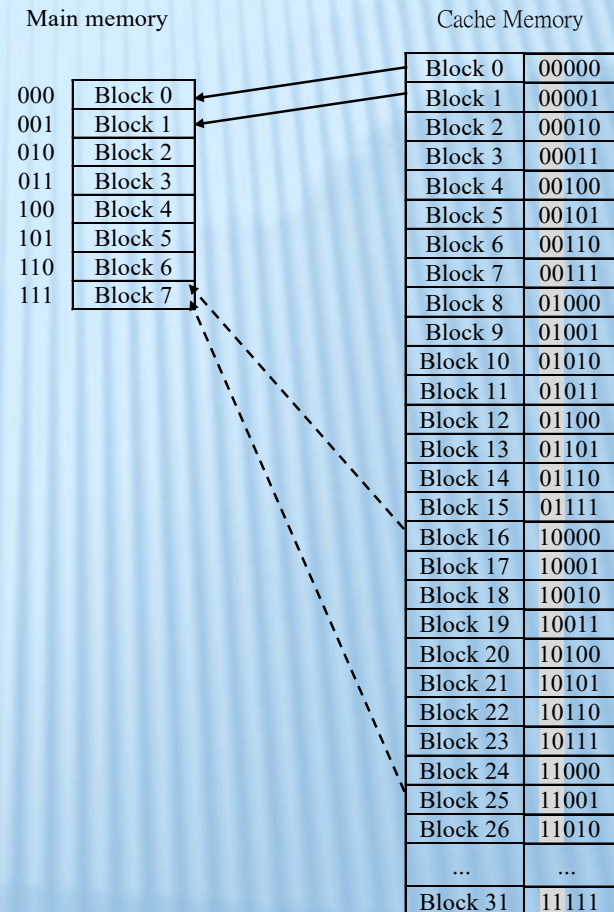
Direct Mapped Cache

✖ Direct reflection cache mechanism :

- + Any main memory block can only be fixed mapped to a cache block.
- + Because the number of blocks in main memory is far greater than the total number of cache blocks, there is a situation where multiple main memory blocks share cache blocks.

Direct Mapped Cache

Direct mapping of main memory blocks to cache blocks: The number of blocks X in the main memory and the total number of cache blocks N are calculated as " $X \bmod N$ " to map directly to the cached block Y .



Direct Mapped Cache

- ✖ For direct mapping, the main memory address is split into three fields :
 - + Word field
 - + Block fields
 - + Label field

Direct Mapped Cache

✖ Word field

- + Also known as the offset field, it is used to determine which word in the block the destination address points to; The length of this field is related to the size of the block and is usually used to represent the relative range of addresses in the block. Its units are bytes, but in a per-word access action, the destination address points only to the first byte of a word.

Direct Mapped Cache

✖ Block fields

- + Each main memory block in the direct mirroring method is mapped to a fixed cache block, and the value of this block field is the cache block number it maps to.

✖ Label field

- + All bits other than the two fields mentioned above belong to the block's label. When a block in main memory is copied to cache memory, the block's label is also stored in the cache block record to indicate the source of the cached block content.

Direct Mapped Cache

- ✖ A partitioned instance of a 16-bit address :
 - + The size of each field is determined by the physical characteristics of the main memory and cache.
 - + 11 bits on the far right: The relative position of the destination address in the target block.
 - + The main memory has 32 blocks, and this part occupies the remaining 5 bits
 - ✖ The 3 bits of the low bit: which cache block is mapped to.
 - ✖ 2 bits on the left: mapped to which main memory block in the same cache block.

label	block	Word Set
2 bits	3 bits	11bits

Direct mapping to the main memory
address field split format

Direct Mapped Cache

- ✖ When the CPU wants to access data in an address, the cache line automatically retrieves the block fields in that address and then queries the mapped cache block records :
 - + If the significant bit is 0, it means that the cache error is incurred, and the autonomous memory will move the block containing the address to the mapped cache block.
 - + If the valid address is 1, then the label of the destination address and the cache block record is matched :
 - ✖ If there is a match, then the address in the word field is used to access the data in the mapped cache block.
 - ✖ If the two do not match, the data currently in the cache block is moved out and the contents of the destination main memory block are moved into the cache block.

Fully-Associative Cache

- ✖ There is no information mapped to the cache block at all in the main memory address, so each main memory block can be placed at any block location in the cache.
- ✖ The only way to find out if the block with the destination address exists in the cache memory is to search the entire cache.
- ✖ Relational cache requires special hardware for relational search.

Fully-Associative Cache

- ✖ The main memory address with a fully relational caching mechanism is only divided into two parts :
 - + Label field
 - + Word field

label	Word Set
5 bits	11 bits

The main memory address format for a fully relational map

Fully-Associative Cache

- ✖ When memory access is made, the CPU must search all cache block records to confirm that the destination main memory block is stored in cache memory.
- ✖ In a fully relational map, one of the blocks must be selected to be removed after the cache is completely filled, making room for new blocks
 - + The replacement algorithm is used to determine which cache block to replace.

Set Associative Cache

✖ Direct projection :

+ Advantage :

- ✖ The caching mechanism is simple and cheap

+ Disadvantage :

- ✖ There can only be fixed location restrictions, and the cache space cannot be fully utilized.

Set Associative Cache

- ✖ Fully related :

- + Advantage :

- ✖ Not limited by a fixed location, fully leverage the speed advantage of cache memory

- + Disadvantage :

- ✖ The complexity of the search process is too high, and complex lines must be used to improve efficiency.

Set Associative Cache

- ✖ The caching mechanism integrated with the advantages of direct reflection and full association :
 - + N-way collection relational cache (N-way set-associative cache)
 - ✖ N : positive integers

Set Associative Cache

- ✖ Divide the cache blocks into multiple sets and then map each main memory address directly to one of the cache block collections according to the direct mirroring mechanism.
- ✖ Each cache block in the cache collection that is mapped may be associated with the source main memory block.
- ✖ All cache block collections are equal in size.

Set Associative Cache

- ✖ 2-socket collective relational cache architecture :

Set Associative Cache

- ✖ 4-way collective relational cache architecture :

Set Associative Cache

- ✖ The size of the label field of the N-channel collection relational cache and the number of associations that each collection can provide affect the implementation of the hardware :
 - + Increasing the number of associations increases flexibility
 - ✖ At the same time, the number of comparators has been increased.
 - ✖ Each cache also costs more bits to store the label fields for comparison.

7.4 Cache memory

- ✖ The 80/20 rule and the regionality of references
- ✖ First acquaintance with cache memory
- ✖ The basic operating mode of cache memory
- ✖ The mapping mechanism of the cache
- ✖ **Replacement strategy**
- ✖ Effective access time and hit rate
- ✖ Write strategy for cache memory

7.4.5 Replacement strategy

✖ Replacement

- + When there is a need to "compete" for the same cache block space, the process of removing the content currently occupying the block space to free up space to store new blocks is called substitution.
- ✖ The direct caching mechanism does not require any special displacement methods. Because the mapped cache block can only be freed up directly for a new main memory block.

7.4.5 Replacement strategy

- ✖ For fully relational cache and collection relational cache, an override strategy is required to decide to sacrifice blocks
 - + victim block
 - ✖ Selected to replace the block of cache memory
 - + replacement policy
 - ✖ The algorithm that determines which block to be replaced.

7.4.5 Replacement strategy

✖ Optimal replacement algorithm :

- + It is expected that the data in the cache will be used quickly.
- + Data that can be expected to be moved out of cache memory will not be used in the future, or will not be used again for a long time.
- + Since ordinary computer systems cannot grasp the temporality of each program's future use of each memory block, it can only be used as a criterion for comparing the quality of other algorithms.

7.4.5 Replacement strategy

- ✖ least recently used(LRU) algorithm :
 - + Use temporal regionality as a consideration for choosing sacrificial blocks.
 - + The main disadvantage is that the system needs to save the access history of each cache block
 - ✖ Requires considerable space to maintain.
 - ✖ Slows down the cache.

7.4.5 Replacement strategy

- ✖ first-in, first out(FIFO)algorithm :
 - + The replaced block, regardless of whether it has been used recently or not, is the block that has been in the cache for the longest time.

7.4.5 Replacement strategy

✖ Random algorithm :

- + Each time, choose one of them as a sacrifice block.
- + There will be no thrashing between LRU and FIFO algorithms.

7.4.5 Replacement strategy

Relevance	2-Way		4-Way		8-Way	
Cache Sizes	LRU	Random	LRU	Random	LRU	Random
16 KB	5.2%	5.7%	4.7%	5.3%	4.4%	5.0%
64 KB	1.9%	2.0%	1.5%	1.7%	1.4%	1.5%
256 KB	1.15%	1.17%	1.13%	1.13%	1.12%	1.12%

Error rate of different combinations of cache memory structures and replacement algorithms

7.4 Cache memory

- ✖ The 80/20 rule and the regionality of references
- ✖ First acquaintance with cache memory
- ✖ The basic operating mode of cache memory
- ✖ The mapping mechanism of the cache
- ✖ Replacement strategy
- ✖ **Effective access time and hit rate**
- ✖ Write strategy for cache memory

7.4.6 Effective access time and hit rate

✖ effective access time(EAT)

- + Average time per access.
- + Used to determine the performance of a hierarchical memory system.
- + The weighted value of the cache memory hit ratio and the relative access time between memory tiers.

7.4.6 Effective access time and hit rate

- ✖ Example: Assuming that the access time of cache memory is 10ns, the access time of main memory is 200ns, and the hit rate of cache memory is 97%, then the average time of each access by the processor in the second-tier memory is :

$$\begin{aligned}\text{Effective access time} &= 0.97 \times (10\text{ns}) + 0.03 \times (200\text{ns}) \\ &= 9.7\text{ns} + 6\text{ns} = 15.7\text{ns}\end{aligned}$$

[TLB Animation - Ultra Visual Enhancement](#)

7.4.6 Effective access time and hit rate

- ✖ The effective access time formula can be generalized to a multi-tier memory system. Taking the second tier as an example, the effective access time formula can be rewritten as follows :

$$\text{Effective access time} = H1 \times \text{Cache1} + (1-H1) \times \text{Cache2} + (1-H2) \times \text{Mem}$$

H1 : L1The hit ratio of cache memory

H2 : L2The hit ratio of cache memory

Cache1 : L1The access time of the cache

Mem : The access time of the main memory

7.4 Cache memory

- ✖ The 80/20 rule and the regionality of references
- ✖ First acquaintance with cache memory
- ✖ The basic operating mode of cache memory
- ✖ The mapping mechanism of the cache
- ✖ Replacement strategy
- ✖ Effective access time and hit rate
- ✖ Write strategy for cache memory

7.4.7 Write strategy for cache memory

- ✖ When there is a write to memory, if only the content in the cache is modified during the write, it will cause the cache to be inconsistent with the data in the main memory (inconsistent) ◦
- ✖ Cache memory write strategy :
 - + Decide when you want to update a block of main memory to the contents of a cache block.

7.4.7 Write strategy for cache memory

- ✖ There are three possible cache write strategies :
 - + write-through
 - + write-back
 - + write-buffer

7.5 Virtual memory

- ✖ The origin of virtual memory
- ✖ Paging
- ✖ Segmentation
- ✖ Pagination segmentation
- ✖ efficiency of pagination
- ✖ Pros and cons of using virtual memory

7.5.1 The origin of virtual memory

- ✖ The main function of virtual memory :
 - + Think of the hard drive as an extension of the main memory, increasing the memory address space used by the program.
- ✖ General implementation :
 - + Paging.
 - + Segmentation
 - + Segmentation + pagination

7.5.1 The origin of virtual memory

- ✖ A term commonly used in pagination :
 - + page frames
 - ✖ The partitioned space or block unit size of main memory (physical memory).
 - + pages
 - ✖ The virtual memory is divided into spaces or blocks, each of which is the same size as the page frame, so that it can be mapped to the main memory. Virtual pages are stored on the hard drive and are not put into physical memory until they need to be executed
 - + paging
 - ✖ The process of copying virtual pages on disk to the main memory page.

The End